

CernVM-FS Filebundles for low-latency data distribution in interactive usecases

May 2026

CHEP 2026, Bangkok, Thailand

Valentin Volkl (valentin.volkl@cern.ch), Georgios Christodoulis (georgios.christodoulis@cern.ch), Jakob Blomer (jakob.blomer@cern.ch)



EP-SFT

Software Frameworks And Tools



CernVM-Filesystem

```
$ wget root.cern/dl/root_v6.36.12-  
linux.tar.gz  
$ tar -xzf root_v6.36.12-linux.tar.gz  
$ source root/bin/thisroot.sh  
$ python -c "import ROOT"
```

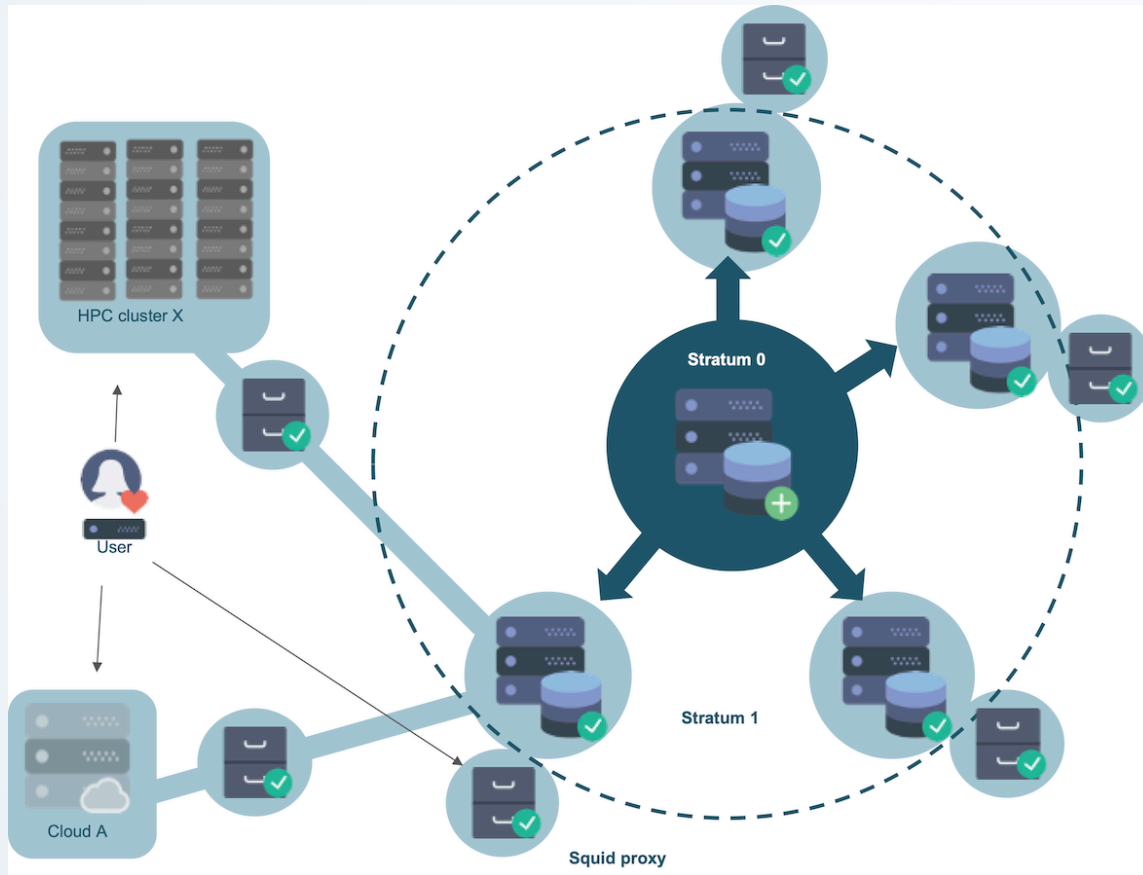
```
$ source /cvmfs/sft.cern.ch/lcg/  
views/  
    LCG_109/x86_64-el9-gcc13-opt/  
setup.sh  
$ python -c "import ROOT"
```

The **CernVM-Filesystem** is an **on-demand, read-only** filesystem for **software distribution**.

Software is fetched transparently from a content-addressed server — only the files actually opened are downloaded.

CVMFS — cache hierarchy

Read-only object store with HTTP fan-out and caching at every hop — every byte is content-addressed, so anything in any cache is reusable forever.



Stratum 0

The origin. One per repo, owned by the librarian.
Where `cvmfs_server` publish writes.

Stratum 1

Globally-distributed HTTP read-only mirrors of Stratum 0. Clients normally talk to the closest one.

Squid proxy

Per-site forward proxy — absorbs duplicate requests when many clients on the same network start the same software.

Client disk cache

Per-node on-disk cache. First open of a file fetches it from the closest upstream tier and writes it here.

Kernel page cache

Linux page cache sits on top of the disk cache — warm reads don't hit `cvmfs2` or the disk at all.

File bundles — Prototype in 2.14, Georgios Christodoulis

Today — one GET per file

python

```
import  
tensorflow
```

↓ `open()` → wait → `open()` → wait ...

GET `tensorflow/__init__.py` → 1 RTT

GET `tensorflow/keras/`
`__init__.py` → 1 RTT

GET `tensorflow/python/eager/`
`context.py` → 1 RTT

GET `numpy/core/`
`_multiarray_umath.so` → 1 RTT

GET `numpy/core/umath.py` → 1 RTT

... × hundreds more

import stalls on each round-trip

With `.cvmfsbundle` — async prefetch

python

```
import  
tensorflow
```

↓ `open(trigger)` → bundle manifest

`.cvmfsbundle.trigger`

async `tensorflow/__init__.py`

async `tensorflow/keras/`
`__init__.py`

async `tensorflow/python/eager/`
`context.py`

async `numpy/core/`
`_multiarray_umath.so`

async `numpy/core/umath.py`

fetches in parallel, in the background

Publishing a bundle — librarian workflow (1/2)

Capture a representative trace, distill it into a spec file.

1

Capture access trace

`CVMFS_TRACEFILE`

Set the option in the cvmfs client config, reload, then run a representative workload. The client logs every `open()` on the repo to a JSON trace.

```
/etc/cvmfs/config.d/my.repo.local
CVMFS_TRACEFILE=/tmp/trace.json

$ cvmfs_config reload my.repo
$ python -c
    'import ROOT; ROOT.TFile()'
```

Trace records every path opened on the repo, with timestamps — enough to derive what should be in a bundle.

2

Generate bundle spec

`.cvmfsbundle-<filename>`

JSON file listing every path the trigger should pull in. Named after its trigger — for `import ROOT` the trigger is `ROOT/__init__.py`, so the spec is `.cvmfsbundle-__init__.py` next to it.

```
$ cvmfs_server trace2bundle
-t /tmp/trace.json my.repo
```

`trace2bundle` is a **proposed** subcommand — today the distillation is done with ad-hoc scripts. Format on the next slide.

Publishing a bundle — librarian workflow (2/2)

Spec format and what `cvmfs_server publish` does with it.

3

Publish

`cvmfs_server transaction / publish`

Drop the spec file next to its trigger and publish. `cvmfs_server publish` sees the `.cvmfsbundle-` prefix and flips the **bundle-trigger** flag on `__init__.py` in the catalog.

```
$ cvmfs_server transaction my.repo
$ cp .cvmfsbundle-__init__.py \
  /cvmfs/my.repo/ROOT/
$ cvmfs_server publish my.repo
```

On disk, one spec per trigger:

```
ROOT/__init__.py ← bundle trigger
ROOT/.cvmfsbundle-__init__.py ← spec (JSON deps)
```

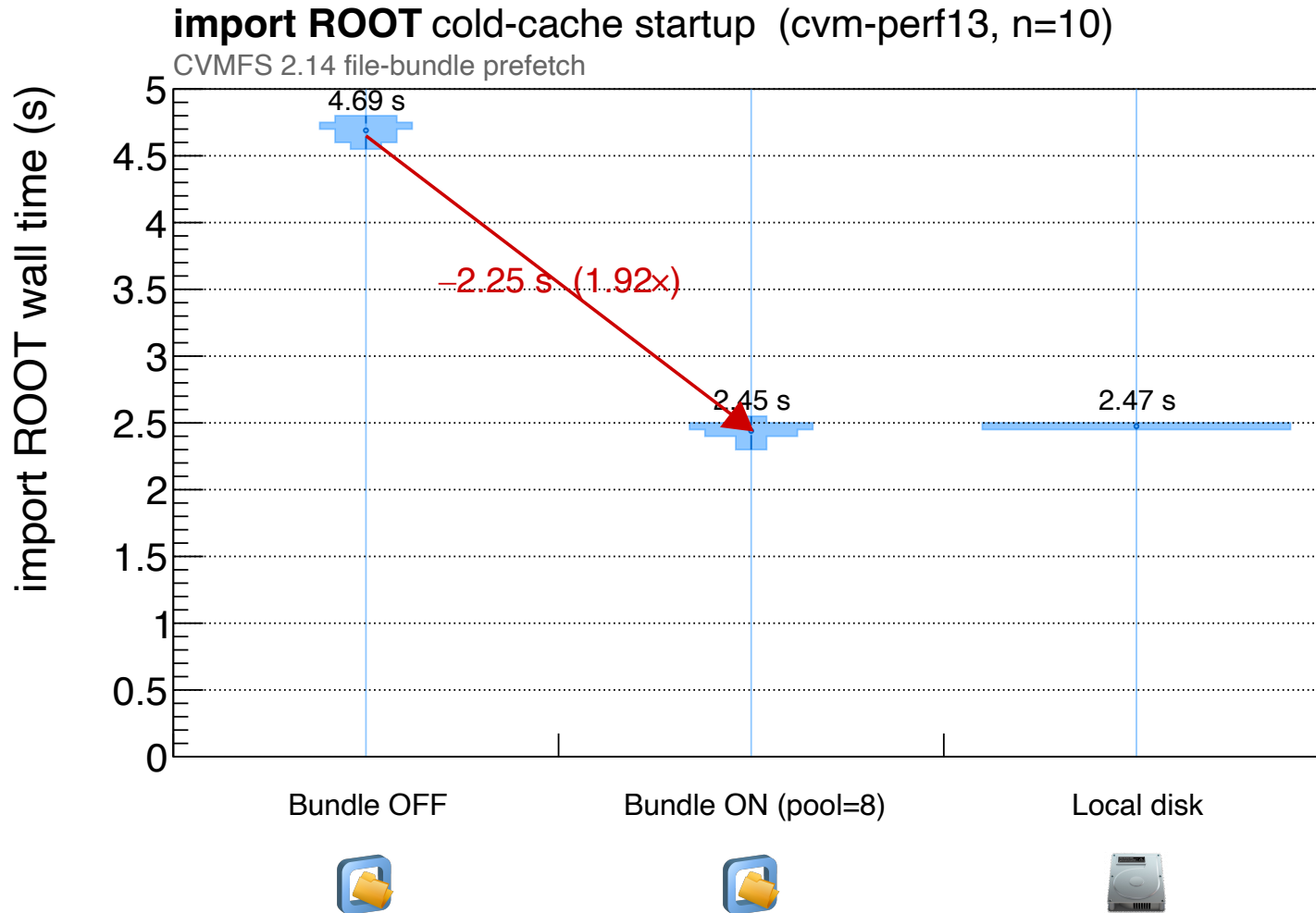
Client `open()` on the trigger → bundle manager loads the spec, fetches every entry in parallel.

Example `ROOT/.cvmfsbundle-__init__.py`

```
#!/CVMFS_BUNDLE version=1 encoding=UTF-8
{
  "name": "CVMFS_BUNDLE",
  "version": "1.0.0",
  "encoding": "UTF-8",
  "dependencies": [
    "/root/lib/libCore.so",
    "/root/lib/libRIO.so",
    "/root/lib/libNet.so",
    "/root/lib/libThread.so",
    "/root/lib/libMathCore.so",
    "/root/lib/libcppy.so",
    "/root/lib/ROOT/_application.py",
    "/root/lib/ROOT/_facade.py",
    "/root/lib/ROOT/_pythonization/__init__.py",
    ... (~250 paths total)
  ]
}
```

Header line carries the format version; the JSON body has a single flat dependencies array of catalog-relative paths.

Filebundle startup performance benchmark



Benchmarking testbed:

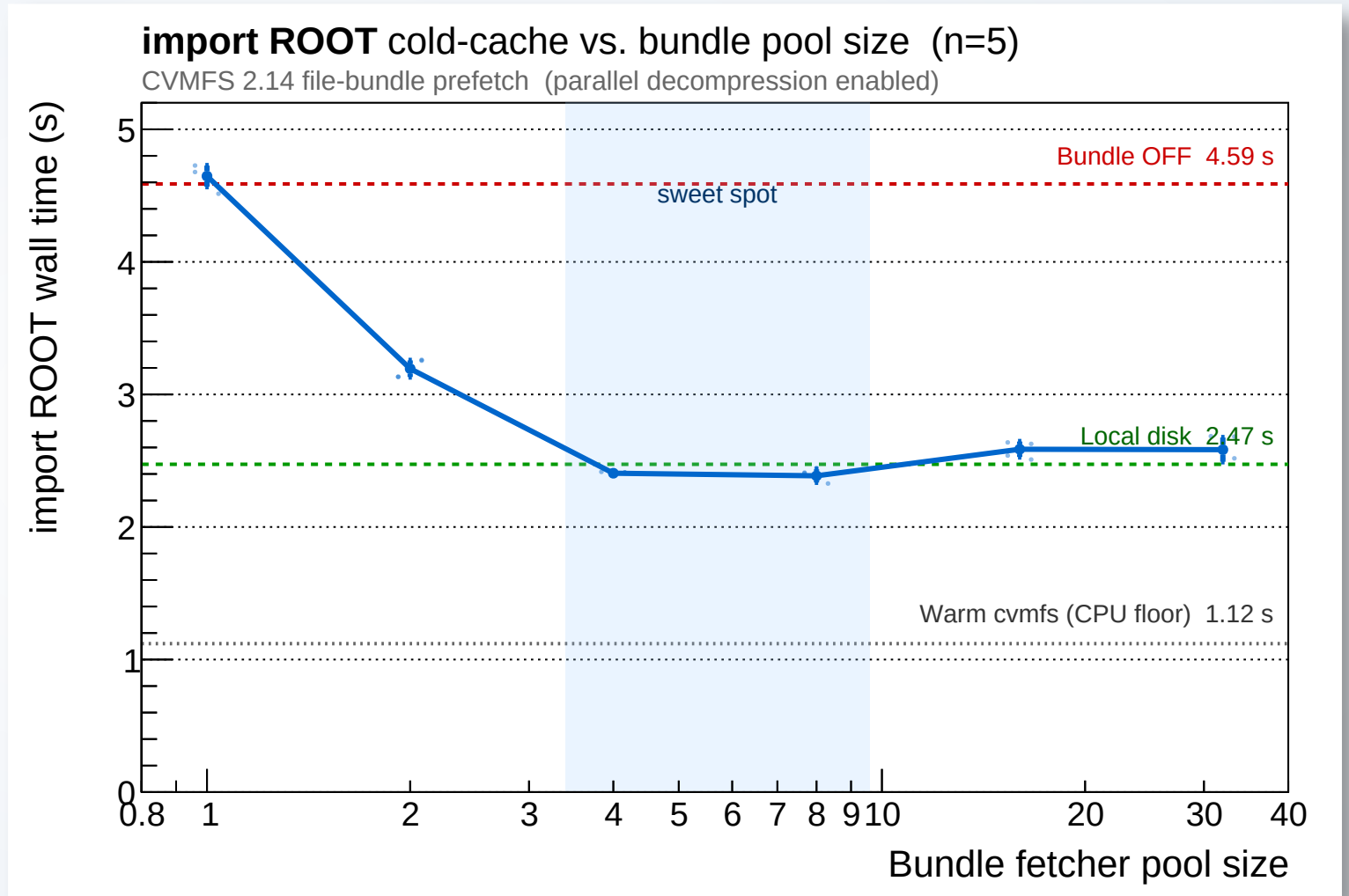
ROOT 6.36.12 deployed on an
test CVMFS repository.
Served on localhost.

AMD EPYC 7302 (2×16 cores /
64 threads @ 3.0 GHz), 251 GB
RAM, AlmaLinux 10.1, kernel 6.12

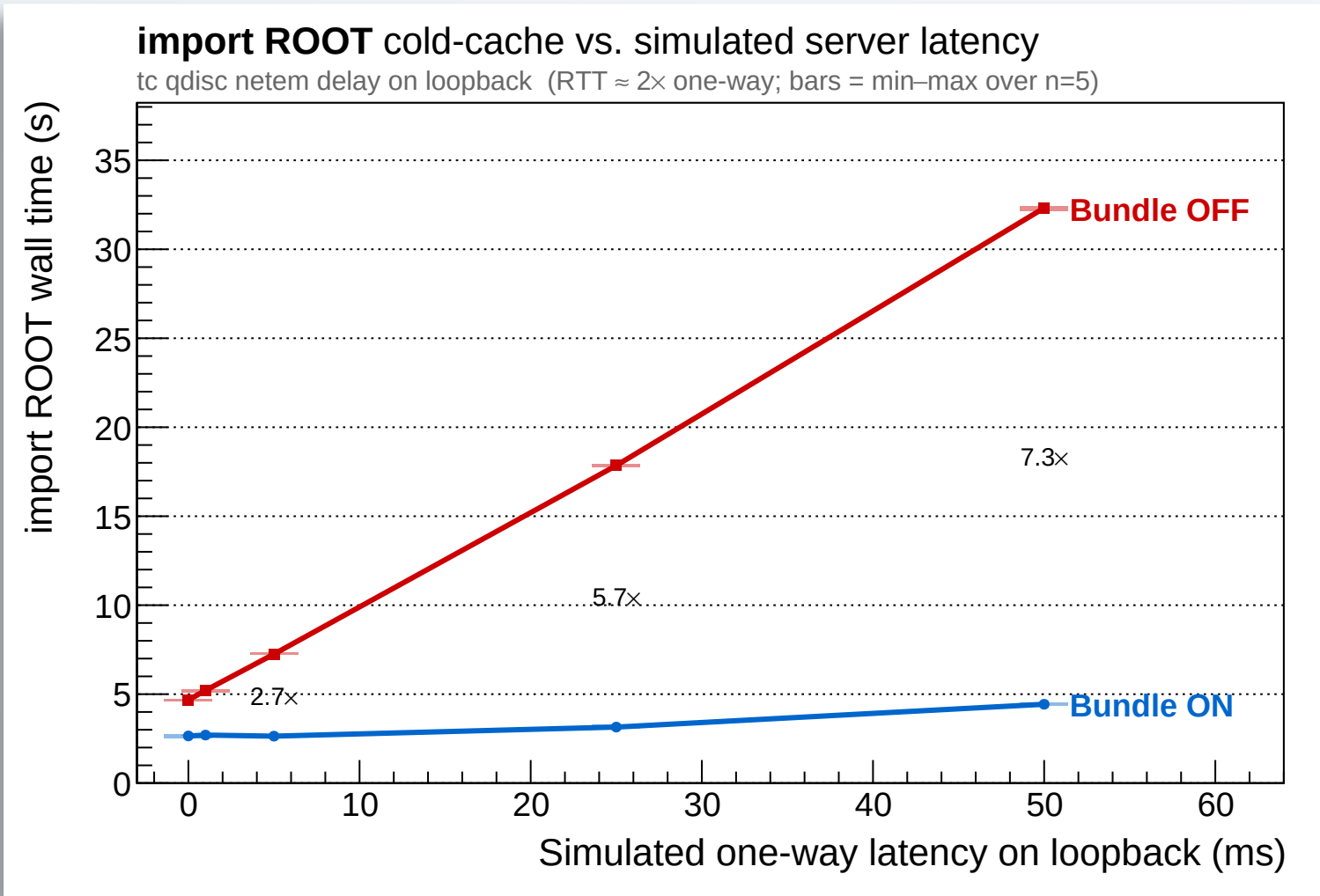
**Matches local-disk
performance.**

Prefetching thread pool

Prefetching employs a pool of worker threads. Size can be set with `CVMFS_BUNDLE_POOL_SIZE`, but saturates after 8 (which is the default).



Filebundles mitigate server network latency



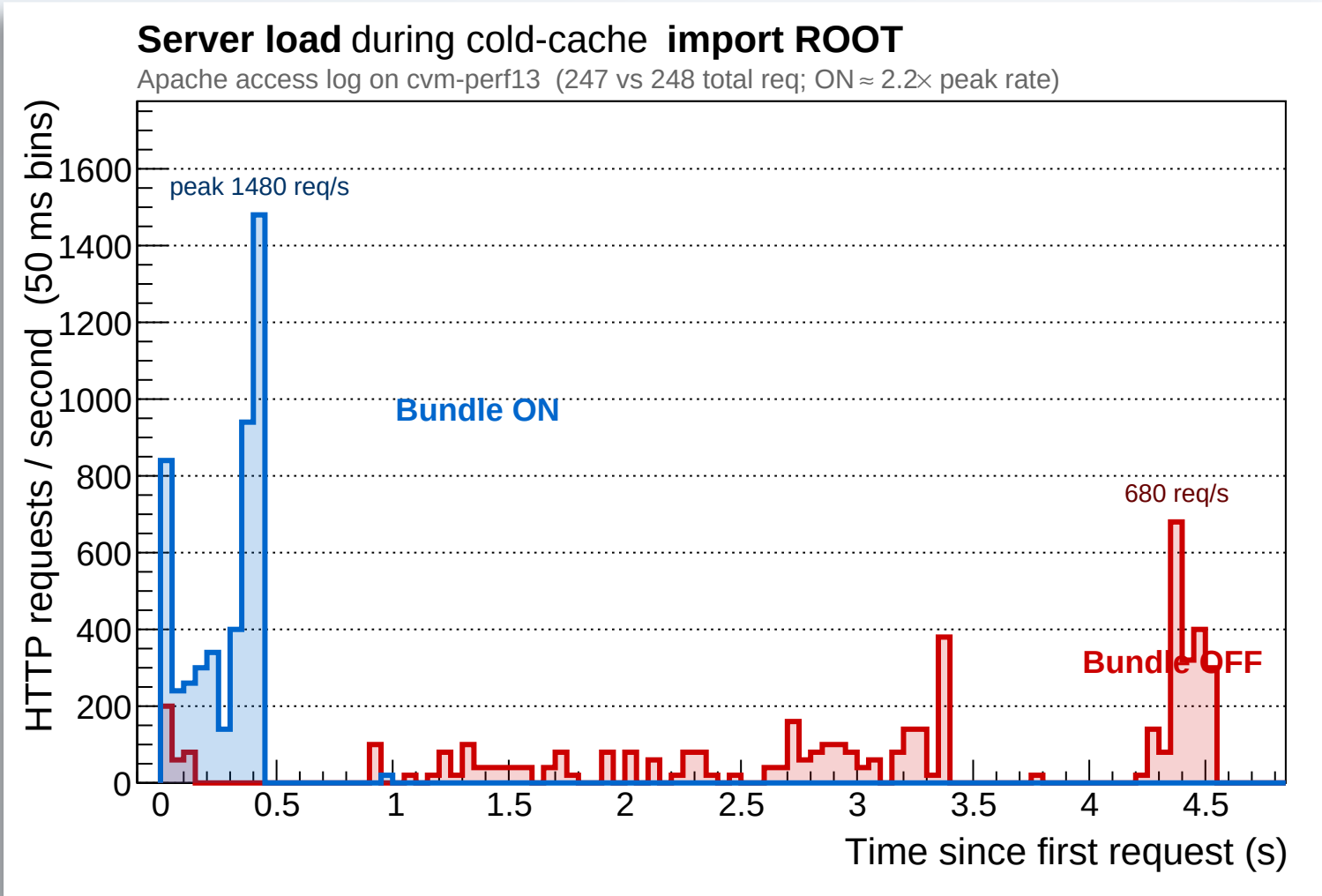
Loopback latency injected artificially with:

```
tc qdisc netem.
```

Startup time without filebundles grows linearly with RTT

ON stays nearly flat in this idealized scenario.

Caveats — bursty load and over-prefetching



Same total downloads — but the `.cvmfsbundle` concentrates the requests into a **0.5 s burst**, peaking at **1480 req/s** vs 680 for OFF.

Implications:

- Should be enabled judiciously, only on interactive clusters like lxplus
- Prefetching more than required further strains the infrastructure
- Rollout should be accorded with proxy admins

Outlook - possible improvements

- **FUSE passthrough (2.14, RHEL10 / kernel $\geq$ 6.9)**

- General performance improvement thanks to new FUSE feature
- On `open()` CVMFS hands the kernel an FD to the backing cache file.
- Subsequent reads bypass FUSE entirely — direct kernel I/O on the cache object.
- Already available in 2.14, but needs to be benchmarked

- **Offline mode / partial persistency in cache (to be seen)**

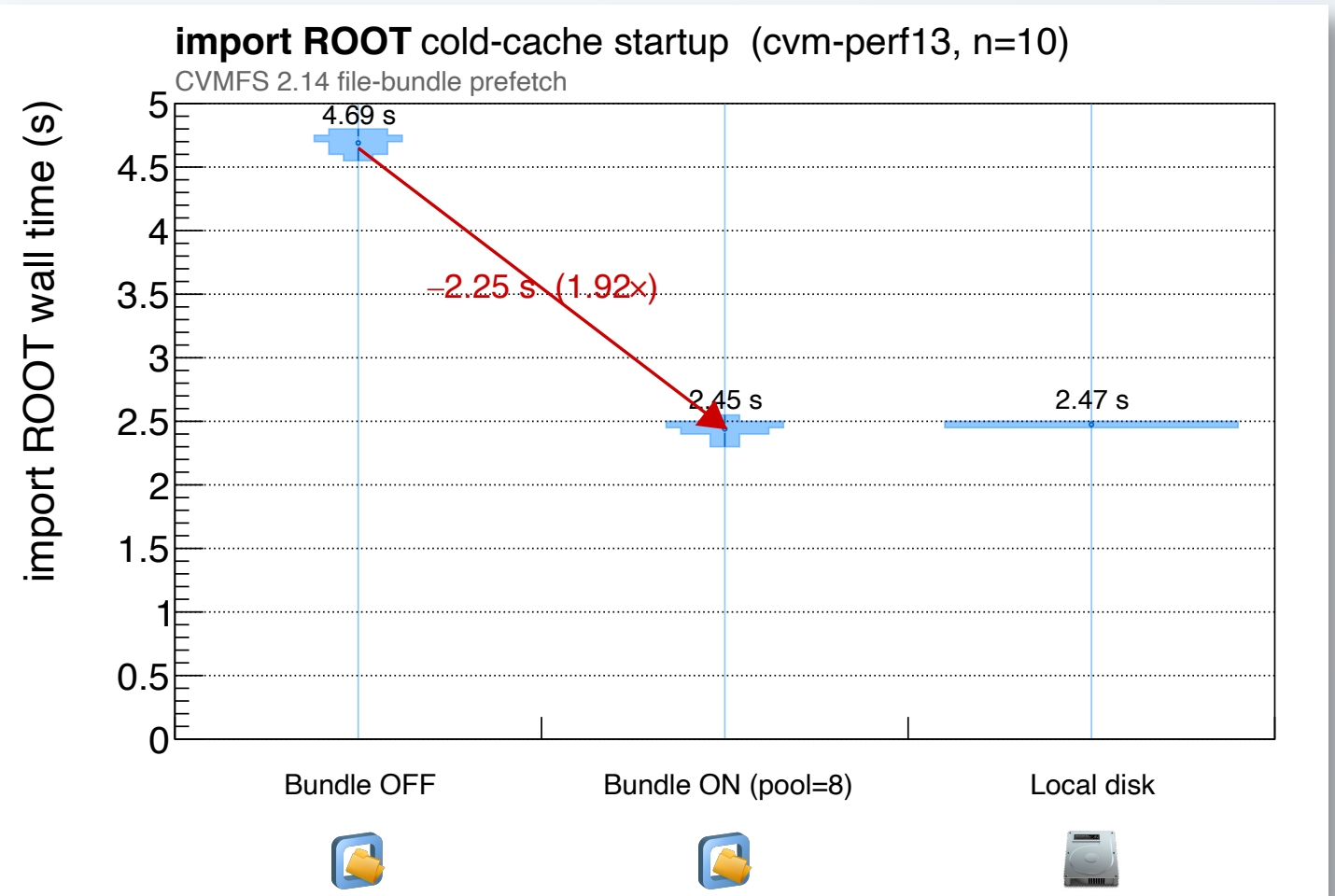
- Give users possibility to partially persist parts of a CVMFS repository on their workstations.
- Integrates with “shrinkwrap” tooling needed for HPC.

- **Packfiles for small files (2.15?)**

- Use packfiles for small objects already in backend storage to limit the number of network requests.
- Backwards compatibility only with significant effort, however

Conclusions

- **Filebundles work.** Cold-cache import ROOT made local-disk performance.
- **One HTTP round-trip** replaces hundreds of sn object fetches — latency, not bandwidth, was the bottleneck.
- **Drop-in for the client** — opt-in via spec, no API changes.
- **Caveat:** bursty server load. Tight specs beat generous ones.
- **Next:** FUSE passthrough, offline mode, packfile



Get in touch

Source code

 github.com/cvmfs/cvmfs

Website & documentation

cernvm.cern.ch

Discussion forum

cernvm-forum.cern.ch

CERN Mattermost

mattermost.web.cern.ch/cvmfs

Thanks! Questions?

Valentin Volkl — valentin.volkl@cern.ch

Stay tuned

CernVM Users Workshop

probably February 2027